Taller 7: Modelado de Hardware en QEMU

José Bernardo Barquero Bonilla
2023150476

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
Curso: CE1113 - Sistemas Empotrados

Prof. Dr.-Ing Jeferson González Gómez

5 de Noviembre de 2026

1. Implementación de `matmul_compute()`

La función implementa el núcleo aritmético del acelerador, $C = A \times B$. Las matrices se almacenan en arreglos lineales, por lo que el elemento $[i][j]$ se ubica en el índice $i \cdot 4 + j$. Antes de multiplicar, cada operando se enmascara con `0xFF`, porque solo los bits [7:0] son válidos. El acumulador es de 32 bits: el valor máximo posible, $4 \times 255^2 = 260\,100$, cabe holgadamente.

```
for (uint32_t i = 0; i < MAT_N; i++) {
    for (uint32_t j = 0; j < MAT_N; j++) {
        s->c[i * MAT_N + j] = 0;
        for (uint32_t k = 0; k < MAT_N; k++) {
            s->c[i * MAT_N + j] += (s->a[i * MAT_N + k] & 0xFFu) *
                                   (s->b[k * MAT_N + j] & 0xFFu);
        }
    }
}
```

En el manejador de lectura, la región C devuelve `s->c[idx]`, con $idx = (addr - \texttt{0x80})/4$ y verificación de rango, y el registro `STATUS` devuelve `s->status`. En el de escritura, el bit `START` de `CONTROL` dispara el cómputo y actualiza `STATUS`.

2. Salida serial de los casos de prueba

El caso 1 es el provisto por la base brindada ($A = 2I$, de modo que $C = 2B$). Para el caso 2 se eligieron matrices con al menos un elemento distinto de cero por fila y columna. Su resultado esperado se calculó de forma externa:

$$\underbrace{\begin{bmatrix} 3 & 1 & 4 & 1 \\ 5 & 9 & 2 & 6 \\ 5 & 3 & 5 & 8 \\ 9 & 7 & 9 & 3 \end{bmatrix}}_{A} \times \underbrace{\begin{bmatrix} 2 & 7 & 1 & 8 \\ 2 & 8 & 1 & 8 \\ 2 & 8 & 4 & 5 \\ 9 & 0 & 4 & 5 \end{bmatrix}}_{B} = \underbrace{\begin{bmatrix} 25 & 61 & 24 & 57 \\ 86 & 123 & 46 & 152 \\ 98 & 99 & 60 & 129 \\ 77 & 191 & 64 & 188 \end{bmatrix}}_{C}$$

En ambos casos el firmware compara la salida del acelerador contra la referencia por software `matmul_sw()` y reporta `PASS`.

```
--- Caso de prueba 1 ---
[HW] Cargando matrices en el acelerador...
[HW] Disparando cómputo (START)...
[HW] Esperando DONE...
[HW] Tiempo de espera START->DONE: 109 us
[HW] Leyendo resultado C...
[HW] C (hardware):
  2  4  6  8
  10  12  14  16
  18  20  22  24
  26  28  30  32

[SW] Calculando referencia en firmware...
[SW] C (software):
  2  4  6  8
  10  12  14  16
  18  20  22  24
  26  28  30  32

[TEST] Comparando resultados...
[TEST] PASS -- todos los elementos coinciden.
```

Figura 1: Caso de prueba 1: tiempo de espera START→DONE medido y resultado `PASS`.

```
--- Caso de prueba 2 ---
[HW] Cargando matrices en el acelerador...
[HW] Disparando cómputo (START)...
[HW] Esperando DONE...
[HW] Tiempo de espera START->DONE: 88 us
[HW] Leyendo resultado C...
[HW] C (hardware):
  25  61  24  57
  86  123  46  152
  98  99  60  129
  77  191  64  188

[SW] Calculando referencia en firmware...
[SW] C (software):
  25  61  24  57
  86  123  46  152
  98  99  60  129
  77  191  64  188

[TEST] Comparando resultados...
[TEST] PASS -- todos los elementos coinciden.
```

Figura 2: Caso de prueba 2: tiempo de espera START→DONE medido y resultado `PASS`, coincidente con la matriz C esperada.

3. Parte 3 – Opción A: latencia simulada

Se modificó el dispositivo para modelar la latencia de un acelerador real con un `QEMUTimer` sobre el reloj virtual de QEMU, con $N = 1000\,\mu$s:

- **instance_init:** se crea el timer con `timer_new_ns()` y se asocia al callback `matmul_latency_cb()`.
- **Escritura de START:** se marca `STATUS = BUSY` y se programa el timer para dentro

de N µs, en lugar de calcular de inmediato.

- **Callback:** al expirar el timer, ejecuta `matmul_compute()` y activa `STATUS = DONE`, lo que equivale al fin de la FSM en hardware.
- **reset:** se cancela con `timer_del()` cualquier cómputo pendiente.

```
static void matmul_latency_cb(void *opaque)
{
    RISCVEduMatmulState *s = opaque;
    matmul_compute(s);
    s->status = STATUS_DONE;
}

case REG_CONTROL:
    if (v & CTRL_START) {
        s->status = STATUS_BUSY;
        timer_mod(s->latency_timer,
                  qemu_clock_get_ns(QEMU_CLOCK_VIRTUAL) +
                  (int64_t)MATMUL_LATENCY_US * 1000);
    }
    break;
```

Las secciones `instance_init` y `reset` estaban marcadas como "NO MODIFICAR", pero la opción A requiere crear y cancelar el timer. Por eso se agregó en ellas una única línea a cada una.

En el firmware, la espera se mide con el contador libre del `riscv-edu-timer` (1 tick $= 1\,\mu$s): se lee el contador justo antes de escribir `START` y justo después de detectar `DONE`, y la diferencia se imprime por UART:

```
t0 = timer_read_counter();
matmul_start();
matmul_wait_done();
t1 = timer_read_counter();
uart_putdec(t1 - t0);
```